

Operações e métodos *em strings*

Strings vêm com dezenas de **métodos** prontos: funções que você chama com um ponto depois do texto. Nesta aula: maiúsculas e minúsculas, limpeza de espaços, busca, substituição, dividir e juntar, verificações como "só tem dígitos?" e alinhamento, fechando com uma validação simples de CPF digitado.

`.upper()` · `.title()`

`.strip()`

`.find()` · `.count()`

`.replace()`

`.split()` · `.join()`

`.isdigit()`

`.center()` · `.zfill()`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Types: String Methods), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

O que é *um método*

Um **método** é uma função que pertence a um tipo. Chama-se com `texto.metodo()`. Como strings são **imutáveis** (aula 07), todo método de string **devolve uma string nova** e deixa a original como estava.

```
nome = "ana"
nome.upper()      # cria "ANA", mas ninguém guardou
print(nome)       # ana
nome = nome.upper() # guarde o resultado
print(nome)       # ANA
```

- ✓ Os parênteses são obrigatórios, mesmo vazios: `.upper()`
- ✓ Métodos podem ser **encadeados**: `texto.strip().lower()`
- ✓ No Cursor, digite o ponto depois de uma string e a lista de métodos aparece
- ✓ No REPL, `help(str.replace)` mostra a documentação

Esquecer de guardar o resultado é o erro número 1 com métodos de string.

Maiúsculas *e minúsculas*

caixa.py

```
texto = "programando em PYTHON"
print(texto.upper())      # tudo maiúsculo
print(texto.lower())      # tudo minúsculo
print(texto.title())      # Primeira Letra De Cada Palavra
print(texto.capitalize()) # Só a primeira letra da frase
print(texto.swapcase())   # inverte maiúsculas e minúsculas
```

SAÍDA

```
PROGRAMANDO EM PYTHON
programando em python
Programando Em Python
Programando em python
PROGRAMANDO EM python
```

- ✓ `.lower()` é a base para comparar sem ligar para maiúsculas:
`resposta.lower() == "sim"`
- ✓ `.title()` ajuda com nomes, mas deixa "Da", "De" com maiúscula também
- ✓ `.capitalize()` deixa só a primeira letra da frase maiúscula

Tirando *o que sobra*

limpar.py

```
email = "    Ana.Souza@Exemplo.com  \n"
print(repr(email.strip()))           # tira espaços das pontas
print(repr(email.lstrip()))          # só da esquerda
print(email.strip().lower())          # encadeando métodos
print("---olá---".strip("-"))         # tira outros caracteres
```

SAÍDA

```
'Ana.Souza@Exemplo.com'
'Ana.Souza@Exemplo.com  \n'
ana.souza@exemplo.com
olá
```

- ✓ `.strip()` tira espaços, tabulações e quebras de linha das pontas
- ✓ `.lstrip()` e `.rstrip()` : só esquerda ou só direita
- ✓ Com argumento, tira outros caracteres: `.strip("-")`
- ✓ `repr()` mostra o texto com as aspas e os `\n` visíveis: ótimo para ver espaços escondidos

Procurando *dentro do texto*

buscar.py

```
frase = "banana com canela"
print(frase.find("na"))      # 2: primeira posição
print(frase.find("xyz"))    # -1: não achou
print(frase.count("a"))     # 5 ocorrências
print(frase.startswith("ban")) # True
print(frase.endswith(".txt")) # False
print(frase.index("com"))    # 7 (find, mas com erro)
```

SAÍDA

```
2
-1
5
True
False
7
```

MÉTODO	DEVOLVE
<code>.find(x)</code>	Posição da 1ª ocorrência, ou -1
<code>.index(x)</code>	Posição, ou ValueError se não achar
<code>.count(x)</code>	Quantas vezes aparece
<code>.startswith(x)</code>	True se começa com x
<code>.endswith(x)</code>	True se termina com x
<code>x in texto</code>	True se x aparece (aula 06)

`.endswith((".jpg", ".png"))` aceita uma tupla de opções: ótimo para checar extensões de arquivo.

Trocando *trechos*

trocar.py

```
frase = "Eu gosto de Java. Java é legal."  
print(frase.replace("Java", "Python"))      # troca todas  
print(frase.replace("Java", "Python", 1))   # só a primeira  
print(frase)                                # a original não muda
```

SAÍDA

```
Eu gosto de Python. Python é legal.  
Eu gosto de Python. Java é legal.  
Eu gosto de Java. Java é legal.
```

- ✓ `.replace(velho, novo)` troca **todas** as ocorrências
- ✓ O terceiro argumento limita quantas
- ✓ Diferencia maiúsculas: "java" não troca "Java"
- ✓ Trocar por `""` apaga: `cpf.replace(".", "")`

Encadear vários `.replace()` resolve muita limpeza de dados, como na validação de CPF da seção 09.

split() e join()

dividir_juntar.py

```
linha = "Ana;29;Recife"
partes = linha.split(";")
print(partes)                # ['Ana', '29', 'Recife']
print(partes[0])
print("o rato roeu a roupa".split()) # sem argumento: espaços
print(" - ".join(partes))          # o contrário do split
```

SAÍDA

```
['Ana', '29', 'Recife']
Ana
['o', 'rato', 'roeu', 'a', 'roupa']
Ana - 29 - Recife
```

- ✓ `.split(sep)` quebra o texto e devolve uma **lista** (aula 09)
- ✓ Sem argumento, separa em qualquer espaço e ignora espaços repetidos
- ✓ `sep.join(lista)` faz o caminho inverso
- ✓ Os pedaços continuam sendo texto: `'29'`, não `29`

Linhas de arquivos CSV, como "Ana;29;Recife", são lidas assim:
`.split(";")` e cada campo pela posição.

Perguntas *sobre o texto*

verificar.py

```
print("2026".isdigit())    # True: só dígitos
print("2026a".isdigit())   # False
print("Ana".isalpha())     # True: só letras
print("Ana Souza".isalpha())# False: o espaço não é letra
print(" ".isspace())       # True
print("ABC".isupper())     # True
```

SAÍDA

```
True
False
True
False
True
True
```

- ✓ Métodos `.is...` respondem `True` ou `False`
- ✓ `.isdigit()` antes de `int()` evita o `ValueError` da aula 04
- ✓ Letras acentuadas contam como letra em `.isalpha()`
- ✓ Espaço não é letra: `"Ana Souza".isalpha()` é `False`

`"-5".isdigit()` e `"2.5".isdigit()` dão `False` : o sinal e o ponto não são dígitos.

Alinhando *e completando*

alinhar.py

```
print("Python".center(20, "*"))
print("Nome".ljust(10) + "|")
print("42".rjust(6) + "|")
print("7".zfill(3)) # 007
print(f{'Produto':<10}{ 'Preço':>10}")
print(f{'Café':<10}{12.5:>10.2f}")
```

SAÍDA

```
*****Python*****
Nome      |
      42|
007
Produto      Preço
Café         12.50
```

- ✓ `.center(n, c)`, `.ljust(n)`, `.rjust(n)` completam até n caracteres
- ✓ `.zfill(n)` completa com zeros à esquerda: códigos e matrículas
- ✓ As f-strings fazem o mesmo com `< > ^` (aula 05)

Validando o formato de um CPF

validar_cpf_simples.py

```
cpf = input("CPF: ").strip().replace(".", "").replace("-", "")
if cpf.isdigit() and len(cpf) == 11:
    formatado = f"{cpf[:3]}.{cpf[3:6]}.{cpf[6:9]}-{cpf[9:]}"
    print("Formato válido:", formatado)
else:
    print("Formato inválido")
```

SAÍDA (DIGITOU " 123.456.789-09 ")

```
CPF: 123.456.789-09
Formato válido: 123.456.789-09
```

- `.strip()` tira os espaços
- Dois `.replace()` tiram pontos e traço
- `.isdigit()` e `len()` conferem 11 dígitos
- Fatias remontam no formato padrão

Isso confere só o **formato**. Um CPF válido de verdade também precisa dos dígitos verificadores, que pedem contas com repetição (aulas 14 e 17). O `if/else` é o assunto da aula 13.

Arrumando *um nome digitado*

Enunciado: peça um nome (que pode vir com espaços e letras misturadas), arrume com `.strip().title()` e mostre o primeiro nome, o último, as iniciais e quantas letras "a" ele tem.

Uma resposta possível

```
# Exercício: arrumar e analisar um nome digitado
nome = input("Nome: ").strip().title()
partes = nome.split()
print("Nome arrumado:", nome)
print("Primeiro nome:", partes[0])
print("Último nome:", partes[-1])
print("Iniciais:", partes[0][0] + partes[-1][0])
print("Quantas letras 'a':", nome.lower().count("a"))
```

SAÍDA (DIGITOU " MARIA DA SILVA ")

```
Nome:      maria DA silva
Nome arrumado: Maria Da Silva
Primeiro nome: Maria
Último nome: Silva
Iniciais: MS
Quantas letras 'a': 4
```

- ✓ `partes[0][0]` : primeira letra do primeiro nome
- ✓ Repare em "Da": o `.title()` não conhece as regras do português
- ✓ Desafio: deixe "da", "de", "do" em minúsculas

Teste o que *ficou*

1. Depois de `nome.upper()` sozinho, nome muda?

A) Sim

B) Não: é preciso guardar o resultado

C) Só a primeira letra

D) Dá erro

B. Strings são imutáveis.

2. `"a,b,c".split(",")` devolve:

A) "abc"

B) ['a', 'b', 'c']

C) ('a', 'b', 'c')

D) 3

B. Uma lista de textos.

3. `"Python".find("z")` devolve:

A) 0

B) None

C) -1

D) ValueError

C. `.index()` é que dá erro.

4. Qual dá True?

A) `"12a".isdigit()`

B) `"-3".isdigit()`

C) `"2026".isdigit()`

D) `"2.5".isdigit()`

C. Só dígitos.

Uma caixa de ferramentas *para texto*

Métodos devolvem uma string **nova**: guarde o resultado. Caixa (`.lower()` , `.title()`), limpeza (`.strip()` , `.replace()`), busca (`.find()` , `.count()`), `.split()` / `.join()` , verificações `.is...()` e alinhamento resolvem a maior parte do trabalho com texto.

Guarde o resultado

`.strip().lower()`

`.split()` · `.join()`

`.isdigit()`

`.zfill()`

Próxima aula → 09 · Utilizando Lista

Fontes: docs.python.org/3/library/stdtypes.html#string-methods, consultado em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.